

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ

по индивидуальному заданию «динамические структуры данных»
по дисциплине «Язык программирования Python»

Вариант 13

Работу выполнил
студент группы ИТ-9 1 курса
Федоров.Д.А.
19.06.2026 г.

Работу проверила
Шилина.А.В.
19.06.2026 г.

Пермь 2026

Содержание

Постановка задачи.....	3
Алгоритм решения.....	4
Тестирование.....	8
Код программы.....	16

Постановка задачи

Контейнеры. На складе хранятся контейнеры с товарами N различных видов. Все контейнеры составлены в N стопок. В каждой стопке могут находиться контейнеры с товарами любых видов (стопка может быть изначально пустой). Автопогрузчик может взять верхний контейнер из любой стопки и поставить его сверху в любую стопку. Необходимо расставить все контейнеры с товаром первого вида в первую стопку, второго вида — во вторую стопку и т.д.

Программа должна вывести последовательность действий автопогрузчика или сообщение о том, что задача решения не имеет.

Входные данные: в первой строке задаётся одно натуральное число N , не превосходящее 500. В следующих N строках описаны стопки контейнеров: сначала записано число k_i — количество контейнеров в стопке, а затем k_i чисел — виды товара в контейнерах в данной стопке, снизу вверх. В каждой стопке вначале не более 500 контейнеров (в процессе переноса контейнеров это ограничение может быть нарушено).

Выходные данные: выведите описание действий автопогрузчика: для каждого действия укажите два числа — из какой стопки брать контейнер и в какую стопку класть. (Обратите внимание, что минимизировать количество операций автопогрузчика не требуется.) Если задача не имеет решения, выдайте одно число 0. Если контейнеры изначально правильно размещены по стопкам, выводить ничего не надо.

Алгоритм решения

Типы данных:

1. Целочисленный тип данных (int), для работы с базовыми математическими операциями.
2. Строковый тип данных (str), для общения с пользователем и обработки ввода.
3. Списки (list), для хранения информации с доступом по индексу.
4. Словари (dict), для создания связи между типом стека и его положением в списке стеков.
5. Множества (set), для удаления дублирующихся элементов.
6. Стеки (StackOfContainers), самописный класс, представляет собой улучшенный для конкретной задачи вариант стандартного стека. Предназначен для хранения и удобного управления контейнерами.
7. Узлы (Node), самописный класс, представляет собой контейнер, хранит в себе тип и ссылку на следующий контейнер.
8. Контекст менеджер (Noisy), самописный класс, аналог @contextmanager, используется для управления режимом вывода.

Описание алгоритма:

Алгоритм разбирает 3 ситуации на каждую из которых предусмотрен свой алгоритм решения:

Терминология: N — количество типов контейнеров, K — количество стопок

1. $N < K$
2. $N = K$
3. $N > K$

Ситуация 1:

1. Первым N стопкам присваиваем свой тип контейнеров, для дальнейшего распределения, далее будем называть эти стопки именованными.
2. Берём $N + 1$ стопку и выгружаем в неё все контейнеры со всех остальных стопок, далее будем называть эту стопку «мусорка».
3. Берём верхний контейнер в мусорке, смотрим какого он типа и перекладываем его в соответствующую именованную стопку.
4. Повторяем шаг 3, пока мусорка не опустеет.

Ситуация 2:

1. Аналогично шагу 1 в ситуации 1, обратите внимание, что именованными будут все стопки, даже мусорки.
2. Берём N стопку и выгружаем в неё все контейнеры со всех остальных стопок, далее будем называть эту стопку «мусорка 1».
3. Берём верхний контейнер в мусорке 1, смотрим какого он типа, если он того же типа, что и мусорка 1, кладём его в первую стопку, далее будем называть её «мусорка 2», если нет, то перекладываем его в соответствующую именованную стопку.
4. Повторяем шаг 3, пока мусорка 1 не опустеет.
5. Берём верхний контейнер в мусорке 2, смотрим какого он типа, если он того же типа, что и мусорка 2, кладём его в любую стопку, не являющуюся мусоркой, такие стопки на данный момент должны быть уже отсортированы, далее будем называть эту стопку «буфер», если нет, то перекладываем его в соответствующую именованную стопку (мусорку 1).
6. Повторяем шаг 5, пока мусорка 2 не опустеет.
7. Берём верхний контейнер в буфере, если он того же типа, что и мусорка 2, то перекладываем его в мусорку 2.
8. Повторяем шаг 7, пока не переложим все контейнеры принадлежащие мусорке 2 из буфера.

Однако для ситуации 2, есть 1 исключение, это ситуация $N = K = 2$, она требует свой отдельный алгоритм и чаще всего для неё не существует решения:

1. Аналогично шагу 1 в ситуации 1.
2. Смотрим верхний контейнер в первой стопке и запоминаем его тип.
3. Берём верхний контейнер в первой стопке и если он того же типа, что и тот, что мы запомнили, то перекладываем его во вторую стопку.
4. Повторяем шаг 3, пока не встретим контейнер другого типа.
5. Если стопки не отсортированы, то смотрим верхний контейнер во второй стопке и запоминаем его тип.
6. Берём верхний контейнер во второй стопке и если он того же типа, что и тот, что мы запомнили, то перекладываем его в первую стопку.
7. Повторяем шаг 6, пока не встретим контейнер другого типа.
8. Если стопки всё ещё не отсортированы, то решения не существует.

Ситуация 3 (самая простая):

1. Для такой ситуации не существует решения.

Архитектура программы и описание модулей

1. Модуль низкого уровня обработки данных (`objects.py`)

Этот модуль отвечает за реализацию структур данных и инкапсуляцию её внутренних свойств.

Класс `StackNode`: Представляет собой элементарный узел динамической структуры. Хранит тип контейнера и ссылку на следующий элемент.

Класс `StackOfContainers`: Управляет последовательностью узлов по принципу LIFO (Last In, First Out).

- Реализует методы `put(value)` для добавления элемента на вершину и `pop()` для извлечения.
- Содержит метод `move(other)`, который совершает перемещение верхнего контейнера из текущего стека в стек `other` с одновременным логированием шага.
- Переопределяет методы `__iter__` и `__reversed__` для последовательного обхода структуры, а также операторы `__add__` и `__iadd__` для слияния стопок контейнеров.

Класс `Noisy`: Контекстный менеджер (реализует протокол `__enter__` / `__exit__`), который динамически управляет флагом `noisy_mode`. Он активирует запись каждого перемещения в файл `output.txt` только на этапе выполнения самого алгоритма сортировки, исключая фиксацию промежуточных действий при инициализации.

2. Модуль логики и алгоритмов (`logic.py`)

Модуль изолирует в себе математическую логику проверки состояний и непосредственного упорядочивания динамических структур. Он не зависит от интерфейса пользователя.

Функция `is_sorted(stacks)`: Проверяет, завершена ли сортировка, анализируя каждый стек на однородность содержащихся в нем элементов (чтобы каждый узел `StackNode` был равен своему следующему `nxt`).

Функция `numerate_stacks(stacks)`: Назначает строковые идентификаторы `stack_id` для объектов стеков (в формате нумерации от 1 до N). Это необходимо для корректного формирования лога перемещений под требования ТЗ.

Функция `sort_containers(stacks, types)`: Главная функция распределения. Реализует вышеописанные алгоритмы.

Вспомогательные утилиты: `clear_console()` и `print_exception(text)` инкапсулируют системные вызовы операционной системы для управления терминалом и визуального отображения задержек (`time.sleep`).

3. Модуль верхнего уровня и интерфейса (`main.py`)

Модуль отвечает за взаимодействие с пользователем, построение интерактивного текстового меню и защиту от некорректного ввода.

Цикл управления: Бесконечный цикл с текстовым меню, обеспечивает возврат пользователя в меню после каждой операции, кроме явного выхода.

Валидация данных: Обёрнута в блоки обработки исключений `try-except ValueError`. Все проверки подробнее расписаны в блоке «Тестирование».

Парсинг входного потока: Использует метод `.split()`, который разбивает строку по любым группам пробельных символов.

Инициализация объектов: На основе входного потока динамически генерирует список объектов `StackOfContainers`, наполняет их узлами и передает данные в модуль `logic.py` для последующей обработки.

Тестирование

Обратите внимание, что выводится в консоль только диалог с пользователем, результат работы программы выводится в файл output.txt, который будет находиться в директории с проектом.

Также очищение консоли не работает в стандартной консоли питона, поэтому программу рекомендуется запускать в терминале или с флагом `emulate terminal in output console`.

Блок 1. Тестирование основных ситуаций.

Тест 1. Ситуация $N < K$.

Входные данные (первое число, это опция):

```
1
3
2 2 1
2 1 2
2 1 1
```

Выходные данные:

```
| 1 | --> | 3 |
| 1 | --> | 3 |
| 2 | --> | 3 |
| 2 | --> | 3 |
| 3 | --> | 2 |
| 3 | --> | 1 |
| 3 | --> | 1 |
| 3 | --> | 2 |
| 3 | --> | 1 |
| 3 | --> | 1 |
```

Состояние стеков:

```
0: 1 1 1 1
1: 2 2
```

Вывод: Тест пройден успешно.

Тест 2. Ситуация $N = K$.

Входные данные:

1

3

2 1 3

2 2 1

2 3 2

Выходные данные:

| 1 | --> | 3 |

| 1 | --> | 3 |

| 2 | --> | 3 |

| 2 | --> | 3 |

| 3 | --> | 1 |

| 3 | --> | 2 |

| 3 | --> | 1 |

| 3 | --> | 1 |

| 3 | --> | 2 |

| 3 | --> | 1 |

| 1 | --> | 3 |

| 1 | --> | 2 |

| 1 | --> | 3 |

| 1 | --> | 2 |

| 2 | --> | 1 |

| 2 | --> | 1 |

Состояние стеков:

0: 1 1

1: 2 2

2: 3 3

Вывод: Тест пройден успешно.

Тест 3. Ситуация $N = K = 2$ (Возможная для решения).

Входные данные:

1

2

3 1 1 2

2 2 2

Выходные данные:

| 1 | --> | 2 |

Состояние стеков:

0: 1 1

1: 2 2 2

Вывод: Тест пройден успешно.

Тест 4. Ситуация $N = K = 2$ (Невозможная для решения).

Входные данные:

1

4 2 1 2 1

2 2 1

Выходные данные:

0

Вывод: Тест пройден успешно.

Тест 5. Ситуация $N > K$.

Входные данные:

1

2

2 3 1

2 2 1

Выходные данные:

0

Вывод: Тест пройден успешно.

Блок 2: Тестирование ошибок пользовательского ввода.

Тест 1. Ввод пользователем неверной опции в главном меню.

Входные данные:

apple

Выходные данные:

```
=====
| Неверная опция, перезапуск программы |
=====
```

Рис. 1. Тип ошибки ввода 1.

Результат: Программа загрузила главное меню заново.

Вывод: Тест пройден успешно.

Тест 2. Ввод пользователем строки или не натурального числа, при указании количества стопок.

Входные данные:

1

0

Выходные данные:

```
=====
| Количество стопок должно соответствовать натуральному числу, |
| перезапуск программы. |
=====
```

Рис. 2. Тип ошибки ввода 2.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 3. Ввод пользователем строки или отрицательного числа в поле ввода контейнеров в любом месте.

Входные данные:

1

3

3 apple pen pen

Выходные данные:

```
=====
| Ошибка ввода, перезапуск программы |
=====
```

Рис. 3. Тип ошибки ввода 3.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 4. Ввод пользователем количества контейнеров в стопке, не соответствующего реальному количеству контейнеров в стопке.

Входные данные:

1

3

1 2 3 4

Выходные данные:

```
=====
| Количество контейнеров указано неверно, перезапуск программы |
=====
```

Рис. 4. Тип ошибки ввода 4.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 5. Ввод пользователем пустого поля в поле ввода контейнеров.

Входные данные:

1

3

2 1 2

2 2 1

«Пустое поле»

Выходные данные:

| 1 | --> | 3 |

| 1 | --> | 3 |

| 2 | --> | 3 |

| 2 | --> | 3 |

| 3 | --> | 1 |

| 3 | --> | 2 |

| 3 | --> | 2 |

| 3 | --> | 1 |

Результат: Программа считала пустое поле, как стек без контейнеров и успешно вывела результат работы.

Вывод: Тест пройден успешно.

Блок 3. Тестирование граничных значений.

Тест 1. Тестирование на минимальном возможном количестве контейнеров в стеке. (По совместительству тест ввода пользователем уже отсортированного стека)

Входные данные:

1

3

1 2

1 1

1 3

Выходные данные: Отсутствуют.

Результат: Программа ничего не вывела т.к контейнеры уже отсортированы.

Вывод: Тест пройден успешно.


```

=====
| Сортировка контейнеров. Пожалуйста, введите необходимую      |
| информацию, в соответствии с шаблоном.                        |
=====
> Введите количество стопок: 3
=====
| Введите контейнеры в соответствии с шаблоном.                  |
| Обратите внимание, что введённые контейнеры будут            |
| расположены в порядке 'снизу-вверх'.                          |
| К - количество контейнеров в стопке,                          |
| а b c ... z - контейнеры определённого типа.                 |
| Если вы хотите оставить стопку пустой, введите 0.            |
| Шаблон: К а b c ... z, Пример: 4 1 2 3 4                      |
=====
> Введите стопку под номером 1: 3 2 1 2
> Ваша стопка 1: 2 1 2
> Введите стопку под номером 2: 3 2 1 1
> Ваша стопка 2: 1 1 2
> Введите стопку под номером 3:

```

Рис. 6. Поля ввода количества стопок и контейнеров.

Код программы

Код программы находится в моём репозитории на github.

Ссылка на репозиторий: https://github.com/BromFD/container_sort